

Cloud-Native DevOps Platform

PART-4

SEETHA LAKSHMI K A P

26-06-2026

Monitoring and Logging Setup using Prometheus, Grafana, Loki

and Alertmanager

1)Objective

To implement a centralized monitoring and logging solution for the cloud-native application using **Prometheus**, **Grafana**, **Loki**, and **Promtail**. This setup enables monitoring of Kubernetes resources, application metrics, ArgoCD metrics, log aggregation, dashboard visualization, and alert generation.

2) Monitoring Configuration Files

2.1) prometheus-values.yaml

Location:

monitoring/prometheus-values.yaml

Purpose:

Configures the kube-prometheus-stack Helm installation.

Important Configurations:

- Grafana administrator username and password.
- Prometheus data retention period (7 days).
- Alertmanager enabled for alert management.

Used During:

Prometheus and Grafana installation.

2.2 loki-values.yaml

Location:

monitoring/loki-values.yaml

Purpose:

Configures the Loki logging stack.

Important Configurations:

- Authentication disabled.
- Amazon S3 bucket used for log storage.
- Log retention using S3.
- Gateway enabled.
- Single replica deployment.
- Cache configuration.

Used During:

Loki installation.

2.3 backend-monitor.yaml

Location:

monitoring/backend-monitor.yaml

Purpose:

Creates a **ServiceMonitor** resource to allow Prometheus to collect metrics from the backend application.

Important Configurations:

- Monitors backend service.
- Scrapes metrics from `/metrics`.
- Collection interval: **30 seconds**.
- Targets the **project-prod** namespace.

2.4 backend-alert.yaml

Location:

monitoring/backend-alert.yaml

Purpose:

Creates a **PrometheusRule** for backend monitoring.

Alert Configured:

- BackendDown

Condition:

Triggers when the backend service remains unavailable for **2 minutes**.

2.5 argocd-servicemonitor.yaml

Location:

argocd/argocd-servicemonitor.yaml

Purpose:

Creates a **ServiceMonitor** to collect ArgoCD metrics.

Important Configurations:

- Monitors ArgoCD metrics endpoint.
- Metrics collection interval: **30 seconds**.
- Enables Prometheus to monitor ArgoCD health and performance.

3. Install Monitoring Stack

Purpose

Install Prometheus, Grafana, Alertmanager and Kubernetes monitoring components using the **kube-prometheus-stack** Helm chart.

Add Helm Repository

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
```

Update Repository

```
helm repo update
```

Create Monitoring Namespace

```
kubectl create namespace monitoring
```

Install Prometheus Stack

```
helm install prometheus prometheus-community/kube-prometheus-stack `
-n monitoring `
-f monitoring/prometheus-values.yaml
```

Verification

```
kubectl get pods -n monitoring
```

```
PS C:\Users\user\Downloads\task-5> kubectl get pods -n monitoring
```

NAME	READY	STATUS	RESTARTS	AGE
alertmanager-prometheus-kube-prometheus-alertmanager-0	2/2	Running	0	3d17h
prometheus-grafana-55c78df9b6-ktc84	3/3	Running	0	3d
prometheus-kube-prometheus-operator-64c7669674-d9fhj	1/1	Running	0	3d17h
prometheus-kube-state-metrics-686cbd6bf6-d6pmv	1/1	Running	0	3d17h
prometheus-prometheus-kube-prometheus-prometheus-0	2/2	Running	0	3d17h
prometheus-prometheus-node-exporter-9ld75	1/1	Running	0	3d17h
prometheus-prometheus-node-exporter-n8626	1/1	Running	0	3d17h
prometheus-prometheus-node-exporter-p66vd	1/1	Running	0	3d17h
prometheus-prometheus-node-exporter-rjjdh	1/1	Running	0	3d17h

```
PS C:\Users\user\Downloads\task-5>
```

4. Install Loki

Purpose

Deploy Loki to collect and store application and Kubernetes logs.

Add Grafana Helm Repository

```
helm repo add grafana https://grafana.github.io/helm-charts
```

Update Repository

```
helm repo update
```

Create Logging Namespace

```
kubectl create namespace logging
```

Install Loki

```
helm install loki grafana/loki `
-n logging `
-f monitoring/loki-values.yaml
```

Verification

```
helm list -A
```

```
kubectl get pods -n logging
```

```
PS C:\Users\user\Downloads\task-5> kubectl get pods -n logging
```

NAME	READY	STATUS	RESTARTS	AGE
loki-backend-0	2/2	Running	0	3d16h
loki-canary-gbsmf	1/1	Running	0	3d16h
loki-canary-hmxb6	1/1	Running	0	3d16h
loki-canary-x5ln8	1/1	Running	0	3d16h
loki-canary-xwzvq	1/1	Running	0	3d16h
loki-gateway-7765d6b456-b6crz	1/1	Running	0	3d16h
loki-read-6cd7c48c58-5cmpg	1/1	Running	0	3d16h
loki-results-cache-0	2/2	Running	0	3d16h
loki-write-0	1/1	Running	0	3d16h

5. Install Promtail

Purpose

Deploy Promtail to collect logs from Kubernetes pods and forward them to Loki.

Install Promtail

```
helm install promtail grafana/promtail `
-n logging
```

Verification

```
kubectl get pods -n logging
```

```
PS C:\Users\user\Downloads\task-5> kubectl get pods -n logging
```

NAME	READY	STATUS	RESTARTS	AGE
loki-backend-0	2/2	Running	0	3d16h
loki-canary-gbsmf	1/1	Running	0	3d16h
loki-canary-hmxb6	1/1	Running	0	3d16h
loki-canary-x5ln8	1/1	Running	0	3d16h
loki-canary-xwzvq	1/1	Running	0	3d16h
loki-gateway-7765d6b456-b6crz	1/1	Running	0	3d16h
loki-read-6cd7c48c58-5cmpg	1/1	Running	0	3d16h
loki-results-cache-0	2/2	Running	0	3d16h
loki-write-0	1/1	Running	0	3d16h
promtail-2sjfp	1/1	Running	0	2d23h
promtail-bptbc	1/1	Running	0	2d23h
promtail-ggq6t	1/1	Running	0	2d23h
promtail-sbs9v	1/1	Running	0	2d23h

```
PS C:\Users\user\Downloads\task-5>
```

6. Verify Helm Installations

Verify all monitoring components.

`helm list -A`

```
PS C:\Users\user\Downloads\task-5> helm list -A
```

NAME	NAMESPACE	REVISION	UPDATED	STATUS	CHART
aws-load-balancer-controller	kube-system	1	2026-06-25 13:32:20.8627419 +0530 IST	deployed	aws-load-balancer-controller-3.4.0
external-secrets	external-secrets-system	1	2026-06-25 14:19:43.3351341 +0530 IST	deployed	external-secrets-2.6.0
loki	logging	1	2026-06-25 17:53:42.5984392 +0530 IST	deployed	loki-7.0.0
prometheus	monitoring	1	2026-06-25 16:25:56.2268039 +0530 IST	deployed	kube-prometheus-stack-87.2.1
promtail	logging	1	2026-06-26 10:24:33.0384142 +0530 IST	deployed	promtail-6.17.1

7. Access Prometheus and Grafana

7.1) Access Grafana

- Forward the Grafana service to the local machine:
`kubectl port-forward svc/prometheus-grafana -n monitoring 3000:80`
- Open the Grafana dashboard:

<http://localhost:3000>

- Login Credentials
Username: admin
Password: admin123

```
PS C:\Users\user\Downloads\task-5\terraform> kubectl port-forward svc/prometheus-grafana -n monitoring 3000:80
Forwarding from 127.0.0.1:3000 -> 3000
Forwarding from [::1]:3000 -> 3000
Handling connection for 3000
Handling connection for 3000
Handling connection for 3000
Handling connection for 3000
Handling connection for 3000
Handling connection for 3000
Handling connection for 3000
Handling connection for 3000
Handling connection for 3000
|
```

7.2) Access Prometheus

- Forward the Prometheus service:
`kubectl port-forward svc/prometheus-kube-prometheus-prometheus -n monitoring 9091:9090`
- Open Prometheus:

<http://localhost:9091>

The screenshot shows the Prometheus web interface with a query of 'up' executed. The results are displayed in a table format, showing the status of various services. The table has two columns: the first column contains the query result (all are 'up') and the second column contains the number of series (all are 1). The table is titled 'Evaluation time' and shows a load time of 1889ms and a result series of 31.

Query Result	Series Count
up{container="node-exporter", endpoint="http-metrics", instance="10.0.12.32:9100", job="node-exporter", namespace="monitoring", pod="prometheus-prometheus-node-exporter-9ld75", service="prometheus-prometheus-node-exporter"}	1
up{container="node-exporter", endpoint="http-metrics", instance="10.0.11.161:9100", job="node-exporter", namespace="monitoring", pod="prometheus-prometheus-node-exporter-p66vd", service="prometheus-prometheus-node-exporter"}	1
up{container="kubernetes-stack", endpoint="https", instance="10.0.11.58:10250", job="prometheus-kube-prometheus-operator", namespace="monitoring", pod="prometheus-kube-prometheus-operator-64c7669674-d9fh", service="prometheus-kube-prometheus-operator"}	1
up{endpoint="https-metrics", instance="10.0.11.161:10250", job="kubernetes", metrics_path="/metrics/cadvisor", namespace="kubernetes", node="ip-10-0-11-161.us-west-1.compute.internal", service="prometheus-kube-prometheus-kubelet"}	1
up{endpoint="http-metrics", instance="10.0.11.203:10249", job="kubernetes", namespace="kubernetes", pod="kubernetes-proxy-62dch", service="prometheus-kube-prometheus-kube-proxy"}	1
up{container="coredns", endpoint="http-metrics", instance="10.0.11.164:9153", job="coredns", namespace="kubernetes", pod="coredns-68466cdd85-xqln4", service="prometheus-kube-prometheus-coredns"}	1
up{container="coredns", endpoint="http-metrics", instance="10.0.12.152:9153", job="coredns", namespace="kubernetes", pod="coredns-68466cdd85-t9oxc", service="prometheus-kube-prometheus-coredns"}	1
up{endpoint="https-metrics", instance="10.0.11.203:10250", job="kubernetes", metrics_path="/metrics/probes", namespace="kubernetes", node="ip-10-0-11-203.us-west-1.compute.internal", service="prometheus-kube-prometheus-kubelet"}	1
up{endpoint="http-metrics", instance="10.0.12.32:10249", job="kubernetes", namespace="kubernetes", pod="kubernetes-proxy-vpx5z", service="prometheus-kube-prometheus-kube-proxy"}	1
up{endpoint="https", instance="10.0.12.103:443", job="apiserver", namespace="default", service="kubernetes"}	1
up{endpoint="https-metrics", instance="10.0.12.83:10250", job="kubernetes", metrics_path="/metrics/cadvisor", namespace="kubernetes", node="ip-10-0-12-83.us-west-1.compute.internal", service="prometheus-kube-prometheus-kubelet"}	1

8. Deploy Monitoring Resources

Purpose

Deploy the monitoring resources required for collecting application and ArgoCD metrics.

Deploy Backend ServiceMonitor

```
kubectl apply -f monitoring/backend-monitor.yaml
```

Deploy Backend Alert Rule

```
kubectl apply -f monitoring/backend-alert.yaml
```

Deploy ArgoCD ServiceMonitor

```
kubectl apply -f argocd/argocd-servicemonitor.yaml
```

Verification

```
kubectl get servicemonitor -A
```

```
kubectl get prometheusrules -A
```

9. Configure Grafana Data Sources

9.1) Purpose

Connect Prometheus and Loki with Grafana to visualize metrics and logs.

9.2) Add Prometheus Data Source

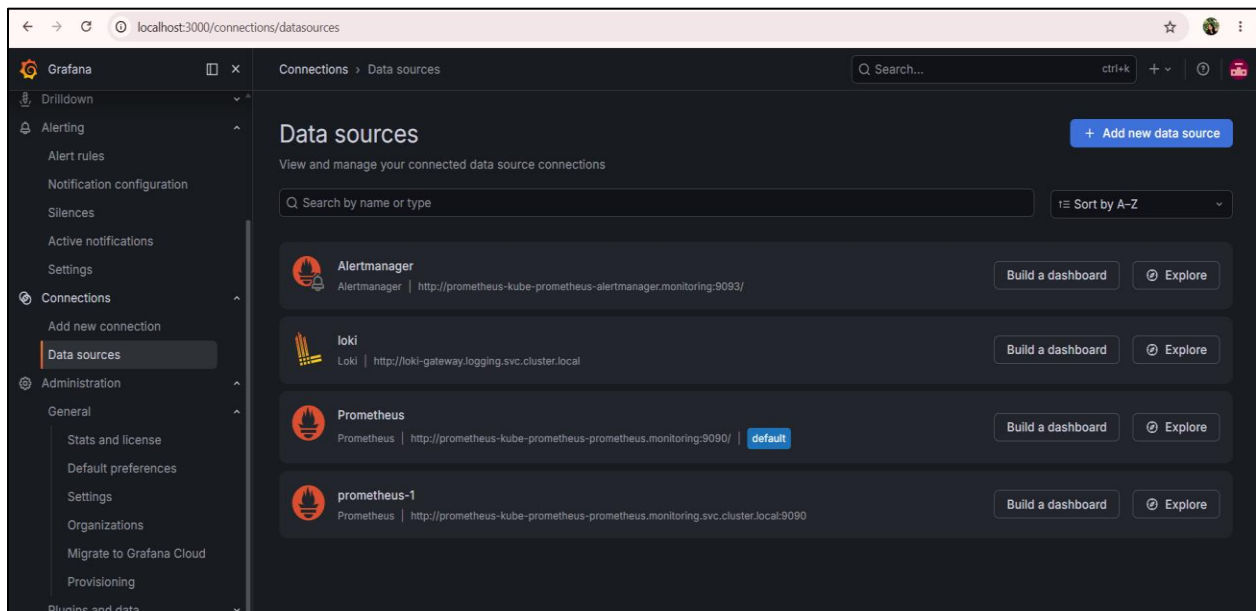
1. Login to Grafana.
2. Navigate to **Connections** → **Data Sources**.
3. Click **Add data source**.
4. Select **Prometheus**.
5. Enter the Prometheus URL:

<http://prometheus-kube-prometheus-prometheus.monitoring.svc.cluster.local:9090>

6. Click **Save & Test**.

9.3) Add Loki Data Source

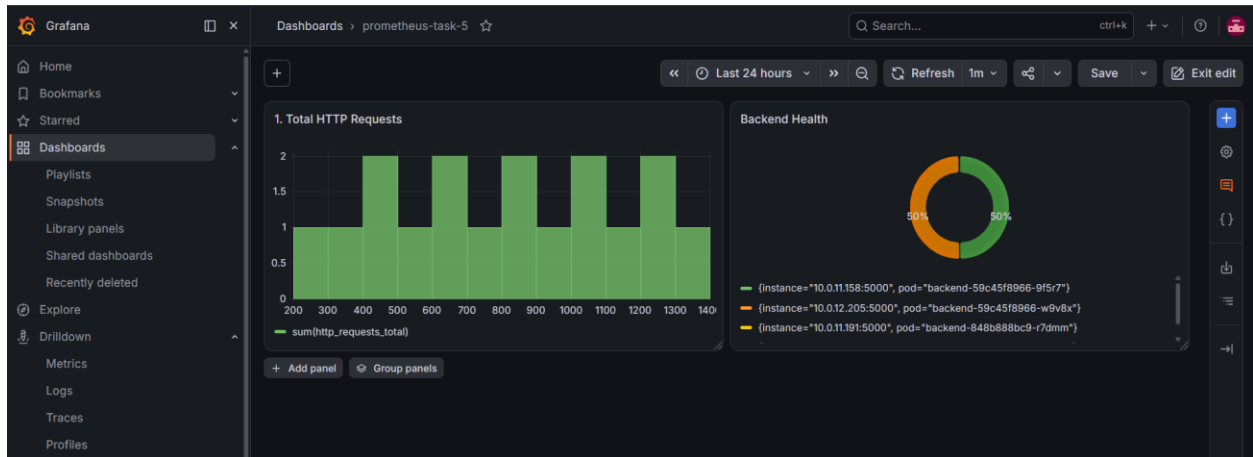
1. Click **Add data source**.
2. Select **Loki**.
3. Enter the Loki URL: <http://loki-gateway.logging.svc.cluster.local>
4. Click **Save & Test**.



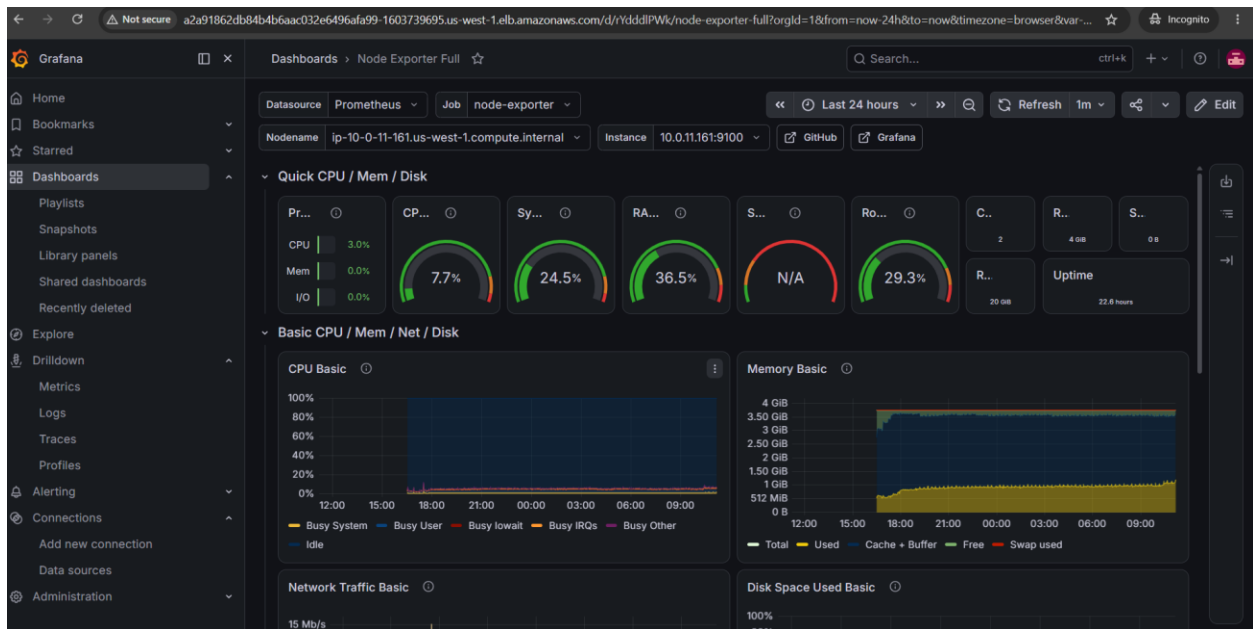
10. Dashboard creation

- Navigate: Grafana → Dashboards → New Dashboard

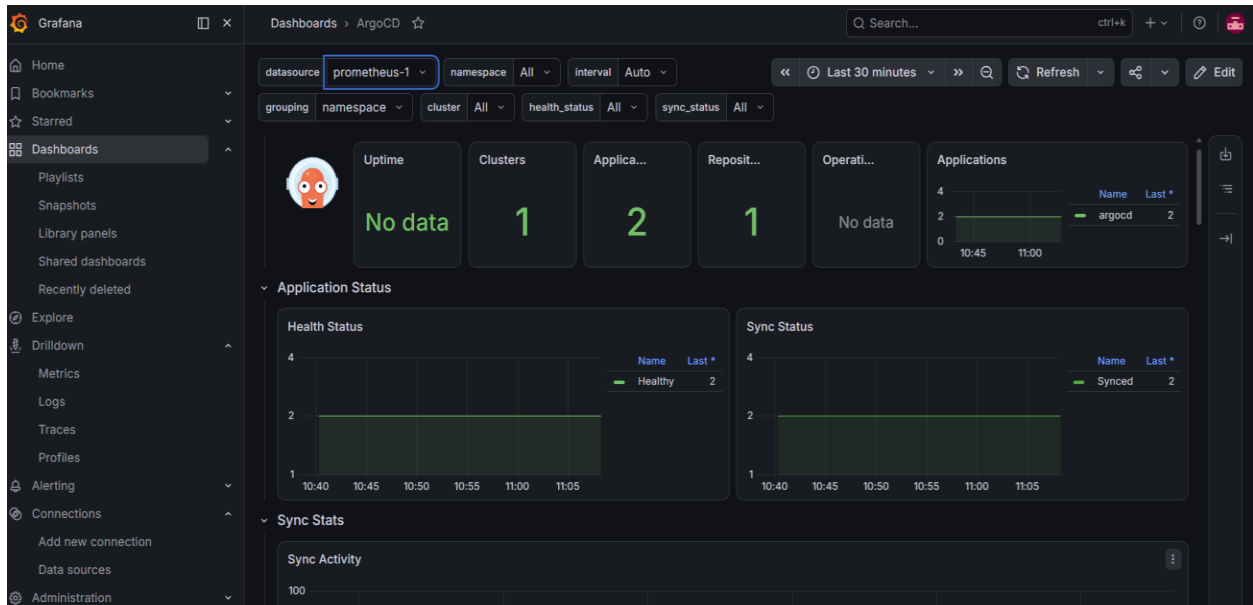
10.1) Prometheus Dashboard



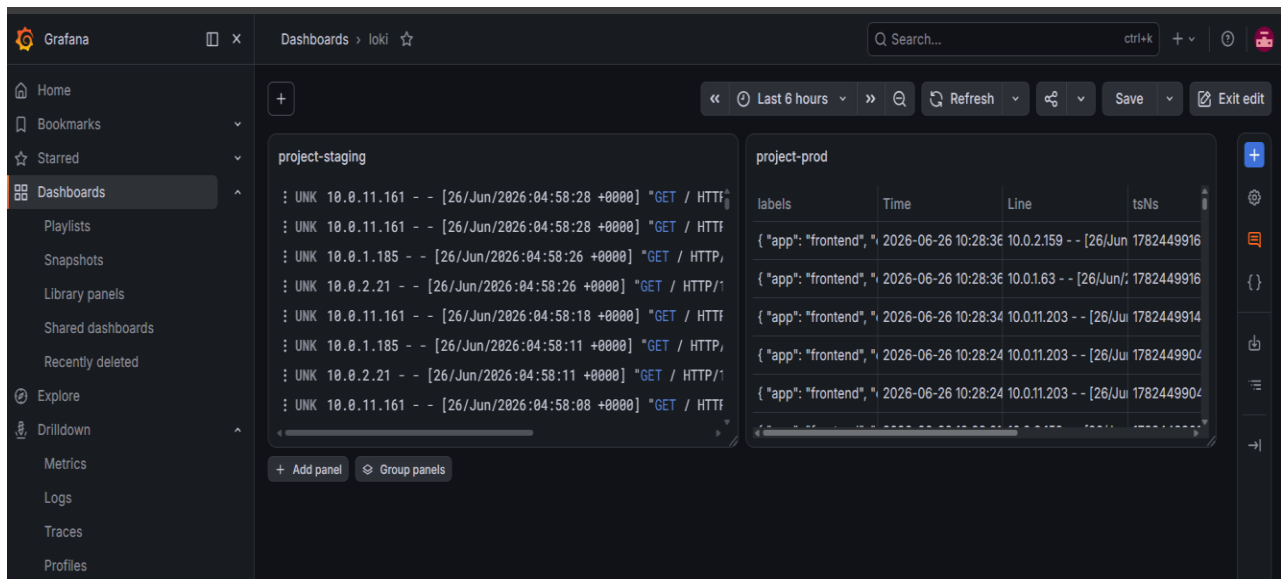
10.2) Node Exporter Dashboard



10.3) ArgoCD Dashboard



10.4) Loki Logs Dashboard



11. Alert Monitoring

Purpose

Monitor backend availability using Prometheus alerts.

Alert Configured

- Backend HTTP Request Alert

Alert Condition

- `sum(http_request_total)`

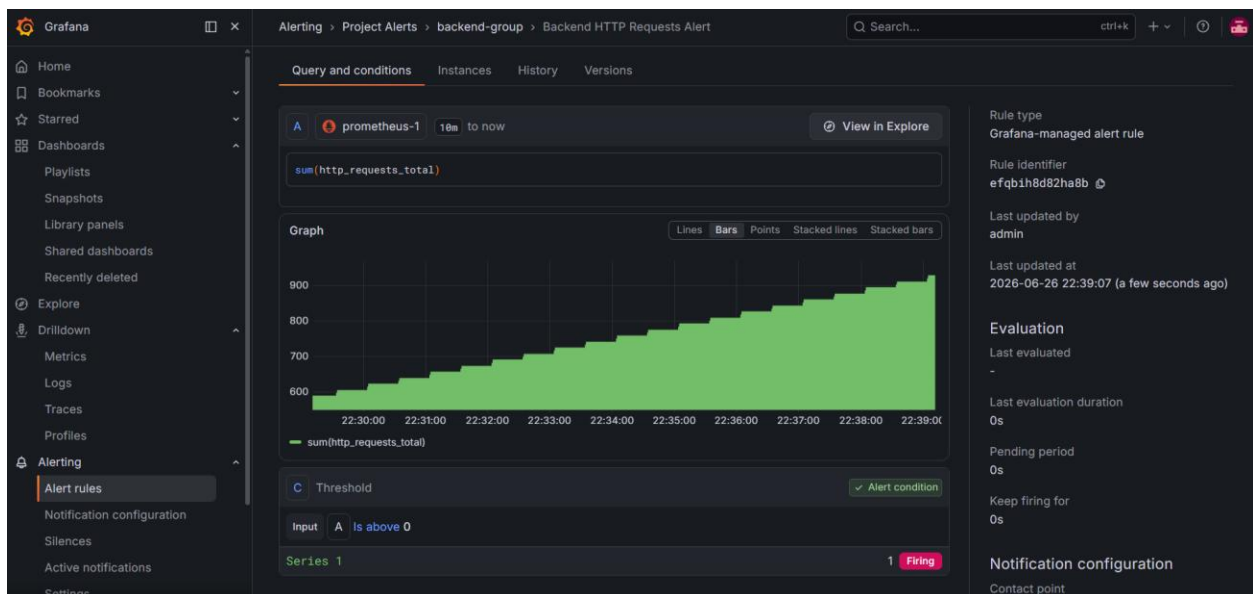
Severity

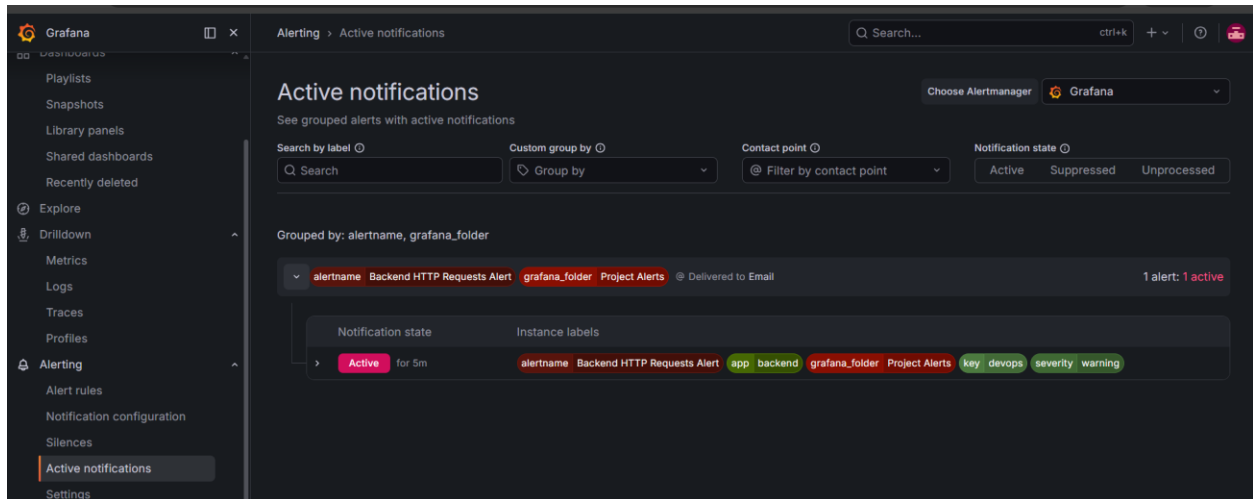
- Critical

Verification

1. Open the Prometheus dashboard.
2. Navigate to **Alerts**.
3. Verify the **BackendDown** alert.

When the backend service remains unavailable for more than two minutes, the alert status changes to **Firing**.





12. Outcome

- Successfully implemented a complete monitoring and logging solution using **Prometheus, Grafana, Loki, and Promtail**.
- Configured Prometheus to collect metrics from the backend application and ArgoCD using **ServiceMonitors**.
- Integrated Prometheus and Loki with Grafana for real-time visualization of metrics and logs.
- Created dashboards to monitor Kubernetes resources, application performance, and ArgoCD metrics.
- Configured alerts to monitor application health proactively.
- Verified monitoring, logging, dashboards, and alerts, providing complete observability for the cloud-native application.